Emojis Aid Social Media Sentiment Analysis: Stop Cleaning Them Out!

Leverage emojis in social media sentiment analysis to improve accuracy.

Max Chen
Jun 2 (10 min read) · Free: Yes*

Photo by Alexandre Debiève on Unsplash

TL;DR:

- Including emojis in the social media sentiment analysis would robustly improve the sentiment classification accuracy no matter what model you use or how you incorporate emojis in the loop
- More than half of the popular BERT-based encoders don't support emojis
- Byte-pair encoding tokenizer performs the best in sentiment analysis and tokenizes well with emojis
- Instead of cleaning emojis out, converting them to their textual description can help boost sentiment classification accuracy and handle the out-of-vocabulary issue.

As social media has become an essential part of people's lives, the content that people share on the internet is highly valuable to many parties. Many modern natural language processing (NLP) techniques have been applied to extracting insights from social media posts. Sentiment Analysis is one of the most popular and critical NLP tasks that focuses on analyzing opinions, sentiments, emotions, or attitudes toward entities in written texts computationally [1]. Social media sentiment analysis (SMSA) is thus a field of understanding and learning representations for the sentiments expressed in short social media posts.

Another important feature of this project is the cute little in-text characters called "emojis". Emojis have become an integral part of general ground in social media communications. According to Emojipedia statistics in 2015, a famous emoji reference site, over one-fifth of all tweets now contains emojis (19.04%), while over half of the comments on Instagram include emojis. Emojis are having and consider ways to express emotions and convey meanings, which may explain their great popularity.

However ubiquitous emojis are in network communications, they are not favored by the field of NLP and SMSA. In the stage of preprocessing, most NLP practitioners have been accustomed to unstructured information like URLs, stop words, unique characters, and pictures [2]. While some researchers have started to study the potential of including emojis in SMSA in recent years, it remains a niche approach and awaits further research. This project aims to **examine the emoji compatibility of trending BERT encoders** and **explore different methods of incorporating emojis in SMSA to improve accuracy**.

Table of Contents

1 **Background Knowledge** 1.1 What is SMSA exactly? 1.2 Development of Sentiment Analysis Methodologies
Experiment 2.1 Model design 2.2 Lesson Learnt in Data Preparation Stage 2.3 Emoji-compatibility Test of the BERT family 2.4 Experimenting Methods to Preprocess Emojis
2 **Results Discussion** 4 **Conclusion Acknowledgments Reference**

1 Background Knowledge

1.1 What is SMSA exactly?

Here is some background knowledge about SMSA you might want to know before looking into the actual experiment. No technical background/math is required so far. Let me first explain the intuition of the most typical SMSA task:

SMSA Example

As the picture above shows, given a social media post, the model should output a vector of three numbers that add up to one for sentiment class. In this example, the model reasoned that this post is 51.62% likely to express positive sentiment, 32.38% likely to be negative, and 16.01% likely to be neutral. Some studies classify posts in a binary way, i.e. positive/negative, but others consider "neutral" as an option as well. This project follows the latter.

1.2 Development of Sentiment Analysis Methodologies

To my best knowledge, the first quantitative approach to studying social media sentiment is using the **lexicon-based method**. The lexicon is a dictionary that maps words to a score that indicates the sentiment score. Its, given a sentence, the model consults the lexicon, **aggregates** the sentiment scores of each word, and outputs the overall sentiment score. Very intuitive, right?

$$\text{Sentence} \longrightarrow \boxed{\text{Functional Lexicon}} \longrightarrow \text{Sentiment Score}$$

Basic Flow of a Lexicon-based Sentiment Analysis Model

SentiStrength and SentiNet are the two paradigms of this kind that have been favored by both the industry and academia.

With the development of **machine learning**, classifiers like SVM, Random Forests, Multi-layer Perceptron, etc., gained ground in sentiment analysis. However, textual input isn't fed into those models, so those classifiers are compounded with **word representation/embedding** algorithms. Word embedding is an embedding models convert words into numerical vectors that machines could play with. Google's word2vec embedding model was a great breakthrough in representation learning for textual data, followed by Stanford's GloVe and Facebook's fastext by Facebook.

Due to the sequential nature of human language and the immense popularity of Deep Learning, **Recurrent Neural Network (RNN)** then becomes "the popular kid" RNN decides, or "reads", one token at a time, mimicking how a human would read. The sequential structure in the loop, which lacks parallelism and traditional machine learning models didn't achieve.

A quick review of word embeddings

The evolved workflow is represented in the diagram above. Word embeddings are passed into an RNN model that outputs the last hidden state, which is then fed into a feed-forward NN that does its intuitively the "summary" contained by the RNN after "reading" all the text. Lastly, we use a fixed-length vector to feed connected neural networks to map the high-dimensional hidden state to a sentiment label.

The paradigm of the **Transformer models**. Even if you haven't learned NLP, you still might have heard about Transformer. In all its hype [3] in this paper, they proposed the self-attention technique and developed a new encoder-decoder architecture that outperformed the best RNN in translation. Transformer is arguably the most famous transformer-based encoder model that learns excellent representations for text. Later on, RoBERTa, BERTweet, DistilBERTa, etc., were developed based on BERT.

2 Experiments

With all those background knowledge, we can now dive into the experiments and programming parts! If you do feel not confident with the mechanism of NLP, I recommend you to skip the technical details or go read some introductory blogs about NLP on Towards Data Science. Let's clarify our experiment objectives first. We want to know:

- How compatible the currently trending pretrained BERT-based models are with emoji data
- How the performance would be influenced if we incorporate emojis in the SMSA process.
- what exactly we should do in the data processing stage to incorporate emojis.

2.1 Model Design

Our model follows the aforementioned neural network paradigm that consists of a pretrained BERT-based encoder, a BiLSTM layer, and a feed-forward fully connected network. The diagram is shown below:

Model Diagram

To be clear, a preprocessed tweet is first passed through the pretrained encoder and becomes a sequence of representational vectors. The representational vectors are then passed through the Bi-LSTM layer. The two last hidden states of the two directions of LSTM will be processed by the feedforward layer to output the final prediction of the tweet's sentiment.

We alter the encoder models and emoji preprocessing methods to observe the varying performance. The Bi-LSTM and feedforward layers are configured in the same way for all experiments in order to control variables. In the training process, we only train the Bi-LSTM and feed-forward layers. The parameters of pretrained encoder models are frozen.

The PyTorch implementation of this model and other technical details can be found in my GitHub Repo.

2.3 Lesson Learnt in Data Preparation Stage

Data availability is every data science researcher's pain in the neck. At first, I wanted to find a benchmark Twitter sentiment analysis dataset where I can compare the results with the previous models, but I encountered the following obstacles:

1. Most datasets only have "tweet ID" as a query key to find the original content. To access the original tweet with the ID, I need to have **Twitter API access**. My professor mentor and I applied for Twitter API access, but neither of us was approved (We still don't know why).
2. Well, another problem is that **a large portion of the tweets already patched**. This means either they were deleted by the author or by the Twitter server because of some reason.
3. Even though there are few datasets that directly store tweet content, those stored in **.csv or .tsv formats are unable to preserve emojis**. I tried to download tweets from a dataset, but the compiled dataset that I downloaded from the web completely lost all emojis.

So, whenever you need to conduct Twitter sentiment analysis, make sure that the dataset you download (if the dataset stores tweets by their Tweet ID, which require you to spend extra effort to retrieve the original text. Tweets can easily perish if the dataset is from years ago. Also, don't expect too much on applying for Twitter API. My mentor, who is an assistant professor of Computer Information Technology, has tried his best (requirement (for some unknown reason). Lastly, to preserve the emojis, don't ever save them in .csv or .tsv format. Pickle, xlsx, or json can be your good choices.

Anyway, to find a **dataset** that retains emojis, has sentiment labels, and is of desirable size was extremely hard for me. Eventually, I found this Novak et al's dataset satisfies all criteria.

2.3 Emoji-compatibility Test of the BERT family

Before implementing the BERT-based encoders, we need to know whether they are compatible with emojis, i.e. whether they can produce unique representations for emoji tokens. More specifically, before passing the tweet into an encoder, it will first undergo tokenization. Some popular tokenizers are used in this study (e.g. RoBERTa uses the RoBERTa-base tokenizer, while BERT-base uses the BERT-base tokenizer). What the tokenizer does is splitting the long strings of text into individual word tokens that are in the vocabulary (shown in the graph below).

Sample Code Output of Tokenizer Tokenizer Vocabulary

In our case, if emojis are not in the tokenizer vocabulary, then they will be tokenized into an unknown token (e.g. "[ ]. Encoder models will thus produce the same vector representation for all those unknown tokens, in which case cleaning or not cleaning out the emojis will technically not make any difference in the model performance.

I chose the following list of common BERT-based encoders:

- ALBERT-base-v2
- BERT-base, BERT-large
- BERTweet-base, BERTweet-large
- DeBERTa-base, DeBERTa-large
- DistilBERT
- RoBERTa-base, RoBERTa-large
- Twitter-RoBERTa
- XLM-RoBERTa-base, XLM-RoBERTa-large

The test can be easily done using the HuggingFace transformers package and the emoji package. We first import them:

```
!pip transformers[sentp] &>/dump/dev/null
!apt emoji
```

AutoTokenizer is a very useful function where you can use the name of the model to load the corresponding tokenizer. The the following one-line code where I import the BERT-base tokenizer:

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

Then, we use the emoji package to obtain the full list of emojis and use the encode and decode function to detect compatibility:

```
emoji_list = [emo[0] EMOJI_DATA.keys()]
# set to speed up
for e in emoji_list:
    sentenceid = tokenizer.encode(tokenizer.encode(e), skip_special_tokens=True)
    if len(sentenceid) == 0:
        continue
    print("UNK token\n\nemoji_list[{}] of the emojis are not identified by the tokenizer".format(emoji_list.index(e)))
```

We came up with 6 ways of data preprocessing methods to make use of the emoji information as opposed to removing emojis (red) from the original tweets.

Directly encode (ALL) Since the pretrained encoders that have been introduced in 2.2 do not contain the emojis in the vocab, emojis are treated as normal word tokens. This is the most straightforward method.

Replacing emojis with descriptions (emojiDesc) The pretrained encoders are not specifically trained to create representations for emoji tokens, they are trained on a vast amount of text, the conjecture that encoders might have better representations for words than emojis, so converting emojis to their official descriptions might help better extract the semantic information. For example, "🥺" → "pleading face". The descriptions testing has with hyper-opens "The python realization is shown below (using the emoji package):

```
def emojiDesc(text):
    return emo2[ replace_emojis(text, replace=lambda chars, data_dict:
```

Concatenate emojis (concat-emoji) Essentially, we reposition the emojis to the end of the tokenized sequence and concatenate them. Since emojis don't belong to the grammatical structure of the sentences, we want to know if repositioning them would help better distinguish the textual and emoji information. For example, "This cat weather is killing me. 😩", Don't wanna work any longer. 😩" → "This cat weather is killing me. Don't wanna work any longer. 😩 😩 😩

Emoji compatibility of BERT-based emoji models

More than half of those models can't recognize all emojis! RoBERTa (both base and large versions), **DeBERTa** (both base and large versions), **BERTweet-large**, and **Twitter-RoBERTa** support all emojis. However, common encoders like **BERT** (both base and large versions), **DistilBERT**, and **ALBERT** nearly do not support any emoji.

Now, let's compare the model performance with different emoji-compatible encoders and different methods to incorporate emojis. The percentage in the following graph indicates the sentiment classification accuracy. Each cell represents the accuracy of an encoder model with a certain preprocessing method.

(Note that emoji2vec is a baseline model that is developed in 2015. It's not BERT-based but it's a predefined emoji-embedding model that can also produce vector representation for emojis. It can be seen as an extension of Google's Word2vec model)

Sentiment Classification Accuracy

To compare different methods to incorporate emojis into the SMSA process, we also show the accuracy across different methods with confidence intervals.

The average accuracy of each preprocessing method (with confidence interval)

One of the most significant insights is that **including emojis, no matter how you include them, enhances the performance of** SMSA **models.** Removing the emojis lowers the accuracy by 1.202% on average. For methods that include emojis, the overlapping confidence intervals indicate a relatively blurry distinction. There's no "generally best" method to utilize emojis.

The average accuracy of each encoder model (with confidence interval)

For comparison among all encoder models, the results are shown in the bar chart above. The confidence interval is also annotated on the top of the bar chart. Small confidence intervals imply **high statistical confidence in the ranking. Twitter-RoBERTa performed the best** across all models, which is very likely caused by the **training domain. emoji2vec**, which was developed in 2015 and prior to the boom of transformer models, holds **relatively poor representations of emojis** under the standards of this time.

Now that no "generally best" method is found, we probe into how different models would benefit differently from various preprocessing methods. The following graph depicts the percentage improvement of using a certain preprocessing method compared with removing emojis at the beginning.

Percentage improvement heatmap

Firstly, all the improvement indices are positive, which strongly justifies the usefulness of emojis in SMSA. Including emojis in the data would improve the SMSA model's performance.

Generally for BERT-based models, directly encoding emojis seems to be a sufficient and sometimes the best method. Surprisingly, the most straightforward methods work just as well as the complicated ones, if not better.

Poor emoji representation learning models might benefit more from converting emojis to textual descriptions. Maximal and minimal improvement both appear on the emoji2vec model. It's likely that emoji2vec has relatively worse vector representations of emojis, but converting emojis to their textual descriptions would help capture the emotional meanings of a social media post.

RoBERTa-large displayed an unexpectedly small improvement regardless of preprocessing methods. Indicating that it doesn't benefit as much from the emojis as other BERT-based models. This result might be explained by the fact that RoBERTa-large's architecture might be more suitable for learning representations for pure text than for emojis, but it still awaits a more rigorous justification.

4 Conclusion

From this project, the key takeaway is that **including emojis in the loop of SMSA would improve the sentiment classification accuracy no matter what model or preprocessing method you use. So, THINK TWICE about cleaning them out when you face a social media sentiment analysis task!**

The best model to handle SMSA tasks and coordinate with emojis is the Twitter-RoBERTa encoder! Please use it if you are dealing with Twitter data and analyzing tweet sentiment.

Regarding how to incorporate the emojis specifically, the methods didn't show a significant difference, so a straightforward way – directly treating the emojis as regular word tokens – would do the job perfectly. Yet, considering that half of the common BERT-based encoders in our study don't support emojis, we recommend using the emoji2vec method. That means converting emojis to their official textual description using a simple line of code (mentioned before, which can easily handle the out-of-vocabulary emoji tokens.

If you are using traditional word embeddings like word2vec and you also don't want to waste the cute emojis, consider using the emoji2vec or concat-emoji method instead of using emoji2vec model.

Hope our project can guide SMSA researchers and industry workers on how to include emojis in the process. More importantly, this project offers a new perspective on improving SMSA accuracy. Diving into the technical bits is not necessarily the only way to make progress, and for example, these simple but powerful emojis can help as well.

Scripts, an academic report, and more can be found in my GitHub Repo.

Regarding images in the post, all unless otherwise noted are by the author.

Acknowledgments

I would like to extend my warmest gratitude to my research supervisor and mentor Professor Mathieu Laurière. He provides me with insightful advice and guides me through this summer research. It is my great honor and pleasure to finish this study with him and receive his email greeting on my birthday.

This work was also supported in part through the NYU IT High Performance Computing resources, services, and staff expertise.

Besides, I genuinely appreciate NYU and NYU Shanghai for offering me the DURF research opportunity.

Thanks to all my friends and family who helped me throughout this summer. The research would not have been possible without any of you.

Reference

[1] Liu, B. Sentiment Analysis: Mining Opinions, Sentiments, and Emotions. (2015), Cambridge University Press, Cambridge.

[2] Chakravarthi, P., Vincent, D. R., Srinivasan, K., Sharma, V., Chang, C.-Y., and Reina, D. G. Emotion-AI-driven Sentiment Analysis: A Survey, Future Research Directions, and Open Issues. (2019), Applied Sciences.

[3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., Kaiser, ω., & Polosukhin, I. Attention is All you Need. (2017), In Advances in Neural Information Processing Systems. Curran Associates, Inc.

[4] Kraj Novak, P., Smailović, J., Sluban, B., & Mozetič, I. Sentiment of Emojis. (2015), PLOS ONE, 10(12), e0144296.

WRITTEN BY

Bale Chen

See all from Bale Chen

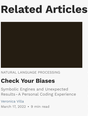
Deep Dives | Emoji | NLP | Sentiment Analysis | Social Media

Share This Article

Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.

Write for TDS

Related Articles



NATURAL LANGUAGE PROCESSING

Check Your Biases

Symbolic Engines and Unexpected Results: A Personal Coding Experience

Veronica Vila
March 11, 2022 • 9 min read

CHATBOTS

Evaluating Cinematic Dialogue - Which syntactic and semantic features are predictive of genre?

This article explores the relationship between a movie's dialogue and its genre using a non-domain-driven data...

Christopher Hanger
January 20, 2024 • 16 min read

MACHINE LEARNING

Large Language Models, MirrorBERT - Transforming Models into Universal Lexical and Sentence...

Discover how mirror augmentation generates data and sees the BERT performance on semantic similarity tasks

Vasishtheo Efthym
December 10, 2023 • 7 min read

NATURAL LANGUAGE PROCESSING

Using NLP and Text Analytics to Cluster Political Texts

NLTK, wolf.nlp.py, or text from Project Guggenheim

Nina Bahadur
January 11, 2022 • 8 min read

NATURAL LANGUAGE PROCESSING

Linguistic Fingerprinting with Python

Attributing authorship with punctuation and syntax

Lee Vaughan
August 1, 2023 • 10 min read

DATA SCIENCE

When Language Meets Data

Our weekly selection of must-read Editors' Picks and original features

TDS Editors
June 1, 2022 • 2 min read

DATA SCIENCE

Natural Language Processing is about More than Chatbots

Our weekly selection of must-read Editors' Picks and original features

TDS Editors
March 3, 2023 • 3 min read

© Insight Media Group, LLC 2026